

Canteen Token / Order Queue System

SDA Project Report

BY

Anees Ahmed: CMS. ID: 053-24-0009

Zuhaib Ahmed: CMS. ID: 053-24-0012

Ghulam Mujtaba: CMS ID:053-24-0023

Abdul Baseer: CMS ID:053-24-0004



Submitted to: SIR BABAR SHEIKH

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF SCIENCE AND INFORMATION TECHNOLOGY

SUKKUR IBA UNIVERSITY

APRIL 27, 2026

Contents

1. Introduction	1
1.1 Purpose.....	1
1.2 Objectives	1
1.3 Scope	1
1.4 Report Organization	1
2. Problem Statement	2
2.1 Background.....	2
2.2 Identified Problems.....	2
2.3 Proposed Solution.....	2
2.4 Objectives	2
3. Requirements Analysis.....	3
3.1 Overview	3
3.2 Stakeholders and User Classes	3
3.3 Functional Requirements	3
3.3.1 User Registration and Authentication.....	3
3.3.2 Menu Management.....	3
3.3.3 Order Placement	3
3.3.4 Token Generation	4
3.3.5 Queue Management.....	4
3.3.6 Order Status Tracking	4
3.3.7 Notifications	4
3.3.8 Order History.....	4
3.3.9 Order Cancellation	5
3.3.10 Admin Dashboard	5
3.4 Non-Functional Requirements.....	5
3.4.1 Performance	5
3.4.2 Security	5
3.4.3 Usability	5
3.4.4 Reliability and Availability.....	5
3.4.5 Scalability.....	6
3.4.6 Maintainability	6
3.5 Out Of Scope.....	6
3.6 Use Case Diagram	6
3.6.1 Actors	6
3.6.2 Main Use Cases	6
3.6.3 Relationship.....	7
3.6.4 Use Case Diagram.....	8

3.7	Use Case Descriptions	9
3.7.1	Place Order.....	9
3.7.2	Update Order Status	10
3.7.3	View Dashboard	11
4.	System Architecture	12
4.1	Architecture Style	12
4.1.1	Diagram	12
4.2	Justification	12
4.3	System Components	13
4.4	Component Interaction	13
4.5	Data Flow Overview.....	13
4.6	Deployment	14
4.6.1	Main Components.....	14
4.6.2	Deployment Structure	14
4.6.3	Communication	14
4.6.4	Description	14
4.6.5	Diagram	15
5.	UML Diagrams	16
5.1	Class Diagram	16
5.2	Sequence Diagram.....	17
5.3	Activity Diagram (Optional)	18
5.4	State Diagram (Optional)	19
6.	Design Patterns	20
6.1	Singleton Pattern.....	20
6.2	Factory Pattern	20
6.3	Observer Pattern	20
7.	Implementation Overview	21
7.1	Technology Stack	21
7.2	Prototype Features	21
7.3	System Structure / Code Organization	22
8.	Testing Strategy	22
8.1	Testing Objectives	22
8.2	Types of Testing.....	23
8.3	Test Cases	24
9.	Conclusion	25

9.1	Summary.....	25
9.2	Constraints and Limitations	25
9.3	Recommendations for Future Work.....	25
10.	References	26
10.1	Books.....	26
10.2	Tools	26
11.	Appendices.....	27
11.1	User Interface	27
11.1.1	Menu	27
11.1.2	Order summary	27
11.1.3	Token generation.....	28
11.1.4	Admin panel	28

1. Introduction

1.1 Purpose

This document presents the Software Design and Architecture of the Canteen Token and Order Queue System. The purpose of this system is to digitize and streamline the food ordering process in a university canteen by replacing manual queuing with a token-based digital order management system. This document covers the complete design decisions, architectural structure, requirements, and implementation strategy of the system and serves as a reference for developers, testers, and academic evaluators.

1.2 Objectives

The key objectives of the Canteen Token and Order Queue System are:

- To eliminate physical queuing at the canteen counter by providing a digital order placement interface
- To generate a unique token for every order so customers can track their position in the queue
- To provide canteen staff with a clear and organized view of pending and ongoing orders
- To notify customers in real time when their order is ready for collection
- To give the admin full control over menu management and order reporting
- To improve overall canteen efficiency and reduce order errors caused by verbal communication

1.3 Scope

The Canteen Token and Order Queue System is a web-based application designed for use within a university campus environment. The system allows students and staff to place food orders digitally, receive a unique token number, and track their order status in real time without physically standing in a queue. The system covers the following:

- Digital menu browsing and order placement
- Automatic token generation and queue management
- Real time order status tracking
- Canteen staff order handling interface
- Admin control panel for menu and report management

The system does not cover the following:

- Online payment integration / Delivery to classrooms or offices
- Inventory or stock management for the canteen

1.4 Report Organization

This report is organized into 11 sections covering the problem statement, requirement analysis, system architecture, UML diagrams, design patterns, implementation overview, testing strategy, conclusion, references, and appendices

2. Problem Statement

2.1 Background

University canteens serve a large number of students and staff during peak hours such as morning breaks, lunch times, and evening breaks. In most universities, the canteen operates manually where customers walk up to the counter, place their order verbally, wait in a physical queue, and collect their food when it is ready. This traditional approach was manageable when the number of users was small, however as university populations grow, this method has become increasingly inefficient and problematic.

2.2 Identified Problems

The following problems have been identified with the current manual canteen system:

- Long physical queues form during peak hours causing significant time waste for students and staff
- Verbal order placement leads to frequent miscommunication and incorrect orders being prepared
- There is no way for a customer to track their order status without standing at the counter
- Canteen staff struggle to manage multiple orders simultaneously without a structured system
- No record of daily orders is maintained making it difficult for admin to analyse demand or manage the menu
- Rush at the counter creates a disorganized and stressful environment for both customers and staff

2.3 Proposed Solution

The proposed solution is a web-based Canteen Token and Order Queue System that digitizes the entire order placement and queue management process. Customers will be able to browse the menu, place their order, and receive a unique token number through the system. The order will automatically enter a digital queue managed on a First in First Out basis. Canteen staff will view and process orders from a dedicated interface and update the order status at each stage. Customers will be notified when their order is ready for collection, eliminating the need to wait at the counter.

2.4 Objectives

The system aims to achieve the following:

- Eliminate physical queuing by shifting the order process to a digital platform
- Reduce order errors through a structured digital menu and order confirmation process
- Provide real time order tracking to customers through token based status updates
- Give canteen staff a clear and prioritized view of all pending orders
- Enable the admin to manage the menu and view order reports efficiently
- Improve the overall canteen experience for students, staff, and canteen workers

3. Requirements Analysis

3.1 Overview

This section describes the functional and non-functional requirements of the Canteen Token and Order Queue System. It also identifies system actors and explains how users interact with the system through use cases.

3.2 Stakeholders and User Classes

- **Customer (Student / Staff)** The primary users of the system. They browse the menu, place orders, receive tokens, and track their order status. They require a simple and fast interface accessible from their mobile or desktop browser.
- **Canteen Staff** The individuals responsible for preparing and serving food. They view incoming orders through a dedicated interface, update order status, and mark orders as ready for collection. They require a clear and real time view of the order queue.
- **Admin (Canteen Manager)** Responsible for managing the system. They add, edit, and remove menu items, view daily order reports, and manage user accounts. They require full control over system content and data.

3.3 Functional Requirements

3.3.1 User Registration and Authentication

ID	Requirement
FR-01	The system shall allow new users to register using their name, university ID, and password
FR-02	The system shall authenticate users through a login page before granting access
FR-03	The system shall maintain separate login roles for customer, canteen staff, and admin

3.3.2 Menu Management

ID	Requirement
FR-04	The system shall display a categorized menu of available food and beverage items
FR-05	The system shall allow the admin to add, edit, and remove menu items
FR-06	The system shall display the name, description, and price of each item

3.3.3 Order Placement

ID	Requirement
FR-07	The system shall allow a logged in customer to select one or more items from the menu
FR-08	The system shall display an order summary before final confirmation
FR-09	The system shall confirm the order and add it to the queue upon customer confirmation

3.3.4 Token Generation

ID	Requirement
FR-10	The system shall automatically generate a unique token number for each confirmed order
FR-11	The system shall display the token number to the customer immediately after order confirmation
FR-12	Token numbers shall reset to 1 at the start of each canteen operating day

3.3.5 Queue Management

ID	Requirement
FR-13	The system shall manage all orders in a First In First Out queue
FR-14	The system shall display the current queue position to the customer
FR-15	The system shall automatically update queue positions as orders are completed

3.3.6 Order Status Tracking

ID	Requirement
FR-16	The system shall track each order through the following statuses: Pending, Preparing, Ready, Collected
FR-17	The system shall display the current status of the order to the customer in real time
FR-18	The system shall allow canteen staff to update the status of each order

3.3.7 Notifications

ID	Requirement
FR-19	The system shall notify the customer when their order status changes to Ready
FR-20	Notifications shall be displayed on the customer's active screen

3.3.8 Order History

ID	Requirement
FR-21	The system shall maintain a history of all past orders for each customer
FR-22	The system shall allow customers to view their previous orders and their statuses

3.3.9 Order Cancellation

ID	Requirement
FR-23	The system shall allow a customer to cancel an order only if its status is still Pending
FR-24	The system shall remove a cancelled order from the queue immediately

3.3.10 Admin Dashboard

ID	Requirement
FR-25	The system shall provide the admin with a dashboard displaying total orders, completed orders, and pending orders for the current day
FR-26	The system shall allow the admin to generate a daily order report

3.4 Non-Functional Requirements

3.4.1 Performance

ID	Requirement
NFR-01	The system shall load any page within 3 seconds under normal network conditions
NFR-02	The system shall support a minimum of 100 concurrent users without performance degradation

3.4.2 Security

ID	Requirement
NFR-03	All user passwords shall be stored in encrypted form using hashing
NFR-04	The system shall use session based authentication to prevent unauthorized access
NFR-05	Each user role shall only have access to features permitted for that role

3.4.3 Usability

ID	Requirement
NFR-06	The system shall be usable by a first time user without any training or manual
NFR-07	The interface shall be responsive and accessible on both mobile and desktop browsers

3.4.4 Reliability and Availability

ID	Requirement
NFR-08	The system shall be available during all canteen operating hours with a minimum uptime of 99%
NFR-09	The system shall handle unexpected errors gracefully without crashing or losing order data

3.4.5 Scalability

ID	Requirement
NFR-10	The system shall be designed to support additional canteen counters or locations in the future without major restructuring

3.4.6 Maintainability

ID	Requirement
NFR-11	The system shall follow a modular design so that individual components can be updated or replaced without affecting the rest of the system

3.5 Out Of Scope

3.6 Use Case Diagram

3.6.1 Actors

- **Customer** – The user who orders food, tracks orders, and manages their account.
- **Canteen Staff** – The person responsible for preparing orders and updating their status.
- **Admin** – The person who manages the system, menu, and reports.
- **Notification System** – Sends alerts/updates to users about their orders.

3.6.2 Main Use Cases

- **UC-01: Register Account**
Customer creates a new account by providing basic details.
- **UC-02: Login**
Users (Customer, Staff, Admin) log into the system using credentials.
- **UC-03: Browse Menu**
Customer views available food items and prices.
- **UC-04: Place Order**
Customer selects items and confirms the order.
- **UC-05: Generate Token**
System generates a unique token number for each order.
- **UC-06: Track Order Status**
Customer checks whether the order is pending, preparing, or ready.
- **UC-07: Cancel Order**
Customer cancels an order before it is completed.
- **UC-08: View Order History**
Customer views their previous orders.
- **UC-09: Update Order Status**
Canteen staff updates order status (e.g., preparing, ready).
- **UC-10: Manage Menu**
Admin adds, updates, or removes food items.

- **UC-11: View Dashboard**
Admin views system overview (orders, activity, etc.).
- **UC-12: Generate Report**
Admin generates reports of sales or orders.
- **UC-13: Send Notification**
System sends updates to customers about their order status.

3.6.3 Relationship

1. Include Relationships (<<include>>)

Include relationships represent mandatory actions that are always executed as part of another use case.

- UC-04 Place Order **includes** UC-05 Generate Token.
- UC-09 Update Order Status **includes** UC-13 Send Notification.
- UC-02 Login **includes** the following use cases:
 - UC-09 Update Order Status
 - UC-10 Manage Menu
 - UC-11 View Dashboard
 - UC-12 Generate Report

2. Extend Relationships (<<extend>>)

Extend relationships represent optional actions that occur under certain conditions.

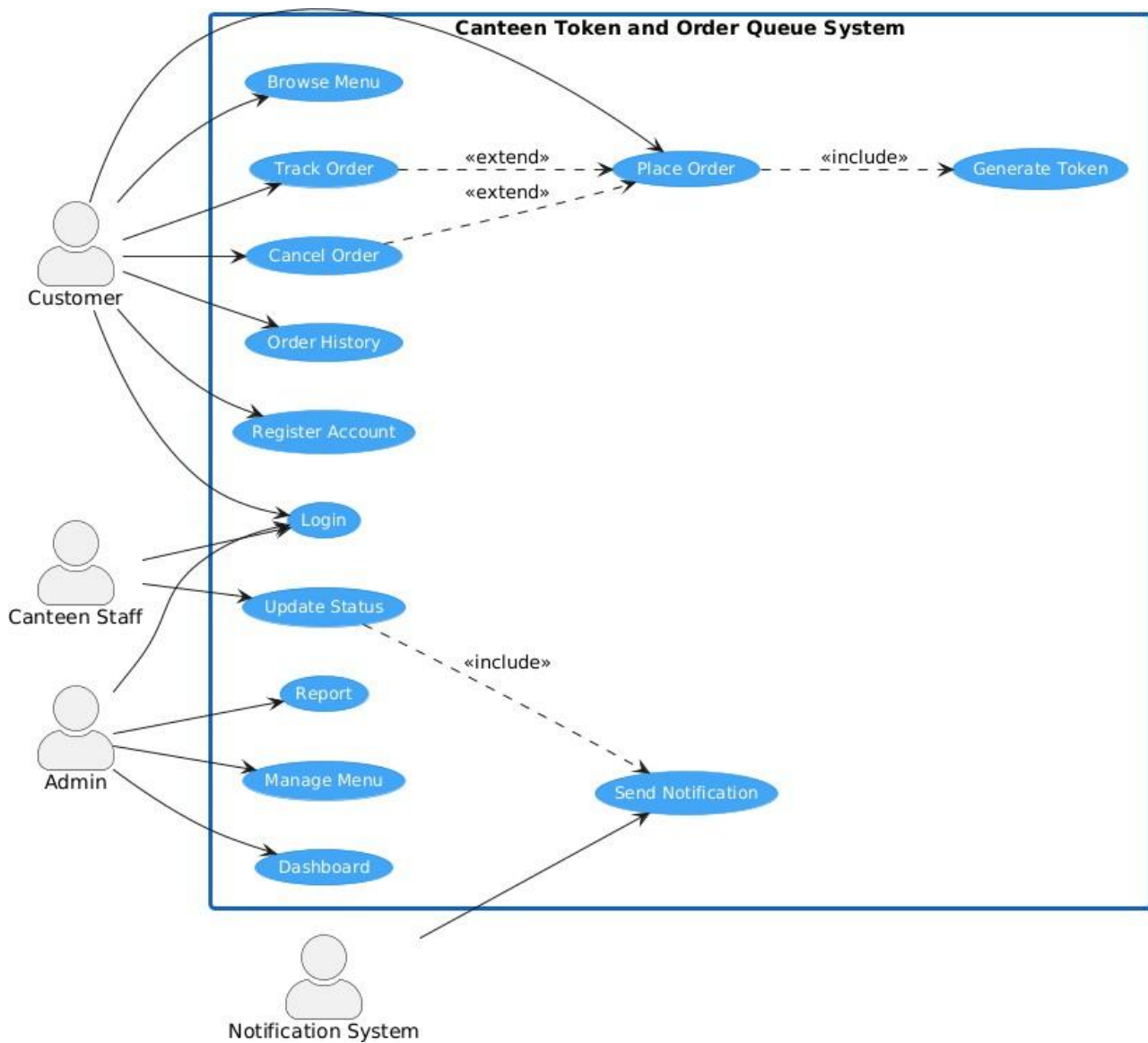
- UC-07 Cancel Order **extends** UC-04 Place Order.
- UC-06 Track Order Status **extends** UC-04 Place Order.
- UC-08 View Order History **extends** UC-02 Login.

3. Association Relationships

Association relationships define the interaction between actors and use cases.

- **Customer** is associated with:
UC-01 Register Account, UC-02 Login, UC-03 Browse Menu,
UC-04 Place Order, UC-06 Track Order Status,
UC-07 Cancel Order, UC-08 View Order History.
- **Canteen Staff** is associated with:
UC-02 Login, UC-09 Update Order Status.
- **Admin** is associated with:
UC-02 Login, UC-10 Manage Menu,
UC-11 View Dashboard, UC-12 Generate Report.
- **Notification System** is associated with:
UC-13 Send Notification

3.6.4 Use Case Diagram



3.7 Use Case Descriptions

3.7.1 Place Order

Customer browses the menu, selects items, and confirms the order which is then added to the queue.

UC-04: Place Order		
Use case Id:		UC-04
Actors:		Customer (Primary User)
Feature:		Order Placement
Pre-condition:		• Customer must be logged into the system • Menu items must be available
Scenarios		
Step#	Action	Software Reaction
1	Customer navigates to menu	System displays available food and beverage items
2	Customer selects one or more items	System updates the order summary
3	Customer reviews order summary	System displays total price and selected items
4	Customer clicks Confirm Order	System adds order to the queue
5	—	System generates a unique token number
6	—	System displays token number to the customer
Alternate Scenarios:		
1a: No items selected → System prompts customer to select at least one item		
2a: Menu is unavailable → System displays "Menu not available at this time"		
Post Conditions		
Step#	Description	
1.	Order is added to the queue	
2.	Unique token number is generated and displayed	

3.7.2 Update Order Status

Canteen staff updates the status of an order as it progresses through preparation.

UC-09: Update Order Status		
Use case Id:	UC-09	
Actors:	Canteen Staff (Primary User)	
Feature:	Order Status Management	
Pre-condition:	• Canteen staff is logged in • Orders are present in the queue	
Scenarios		
Step#	Action	Software Reaction
1.	Staff navigates to order queue	System displays list of pending orders
2.	Staff selects an order	System displays order details
3.	Staff updates status to Preparing	System updates order status and notifies customer
4.	Staff updates status to Ready	System updates order status and sends ready notification
5.	Staff marks order as Collected	System finalizes the order and removes it from active queue
Alternate Scenarios:		
1a: Order already marked Collected → System disables further status update		
Post Conditions		
Step#	Description	
1.	Order status is updated in the system	
2.	Customer is notified of the status change	

3.7.3 View Dashboard

Admin views a summary of daily orders and system activity.

UC-11: View Dashbaord		
Use case Id:		UC-11
Actors:		Admin
Feature:		View Dashboard
Pre-condition:		Admin must be logged into the system
Scenarios		
Step#	Action	Software Reaction
1.	Admin logs into the system	System loads the admin dashboard
2.	—	System displays total orders for the day
3.	—	System displays pending and completed order counts
4.	Admin clicks Generate Report	System compiles and displays the daily order report
5.	Admin logs into the system	System loads the admin dashboard
Alternate Scenarios:		
1a: No orders for the day → Dashboard displays zero counts for all fields		
Post Conditions		
Step#	Description	
1.	Admin has a clear overview of daily canteen activity	
2.	Report can be saved or printed if required	

4. System Architecture

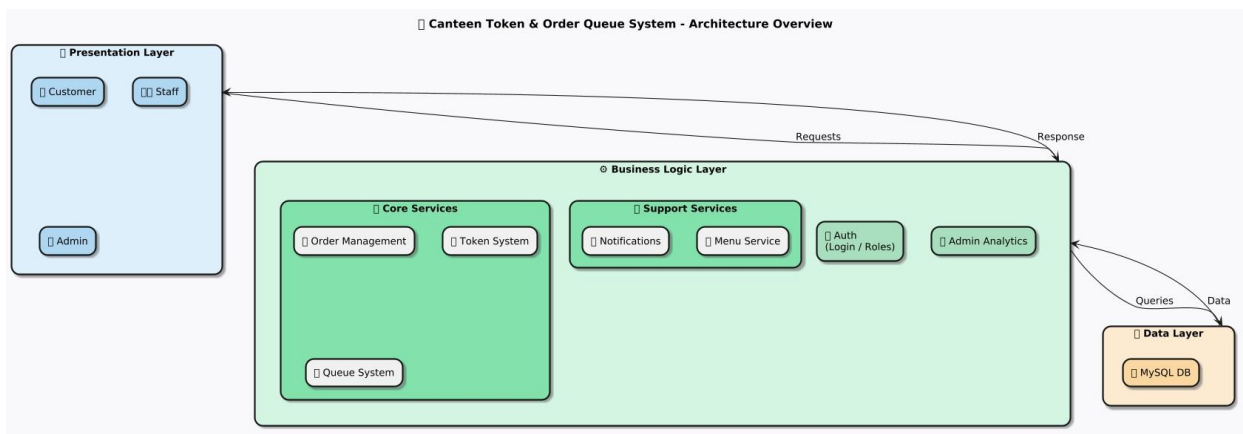
4.1 Architecture Style

The Canteen Token and Order Queue System follows a Three-Tier Architecture. This architectural style divides the system into three distinct layers, each with a clearly defined responsibility, ensuring separation of concerns, maintainability, and scalability.

The three tiers are:

- **Presentation Layer (Tier 1)** — The user interface layer that the customer, canteen staff, and admin interact with directly through a web browser
- **Business Logic Layer (Tier 2)** — The application server layer that processes requests, applies system rules, manages the order queue, and controls token generation
- **Data Layer (Tier 3)** — The database layer that stores and retrieves all system data including users, orders, menu items, and tokens

4.1.1 Diagram



4.2 Justification

The Three-Tier Architecture is selected because it is simple, easy to understand and well-suited for small-scale systems like the Canteen Token and Order Queue System that require clear separation between the user interface, business logic, and data storage. It allows independent development and maintenance of each layer, which makes the system more flexible and easier to update.

This approach also improves system reliability, as issues in one layer do not directly affect other layers. It is particularly suitable for this project because the system is simple and mainly based on order processing and data storage operations.

4.3 System Components

The system is composed of the following major components:

- 1) **Authentication Module:** Handles user registration, login, session management, and role based access control for customers, canteen staff, and admin.
- 2) **Menu Module:** Manages the display of food and beverage items to customers and provides the admin with the ability to add, edit, and remove menu items.
- 3) **Order Management Module:** Handles order placement, order confirmation, order status updates, and order cancellation. This is the core module of the system.
- 4) **Token Generation Module:** Automatically generates a unique sequential token number for each confirmed order. Token numbers reset daily at the start of each canteen operating day.
- 5) **Queue Management Module:** Maintains the FIFO order queue, tracks queue positions, and updates positions as orders are completed by canteen staff.
- 6) **Notification Module:** Monitors order status changes and triggers on-screen notifications to the customer when their order status changes to Ready.
- 7) **Admin Dashboard Module:** Provides the admin with a real time overview of daily orders, pending counts, completed counts, and report generation.

4.4 Component Interaction

The components interact in the following way:

- The customer interacts with the **Menu Module** to browse items and with the **Order Management Module** to place an order
- Upon order confirmation the **Order Management Module** calls the **Token Generation Module** to assign a token and the **Queue Management Module** to add the order to the queue
- The canteen staff interacts with the **Order Management Module** to update order status
- Every status update triggers the **Notification Module** to alert the customer
- The **Admin Dashboard Module** reads data from the **Order Management Module** and **Menu Module** to display statistics and reports
- All modules communicate with the **Data Layer** through the backend for storing and retrieving data

4.5 Data Flow Overview

The general data flow of the system is as follows:

1. Customer logs in through the Presentation Layer and browses the menu
2. Customer places an order which is sent as a request to the Business Logic Layer

3. The Business Logic Layer validates the order, generates a token, and adds the order to the queue
4. The order data is saved to the Data Layer through the database
5. Canteen staff views the queue from their interface and updates the order status
6. The status update is processed by the Business Logic Layer and saved to the Data Layer
7. The Notification Module detects the status change and pushes a notification to the customer
8. The Admin Dashboard reads aggregated order data from the Data Layer and displays it in the dashboard

4.6 Deployment

The deployment diagram illustrates how the Canteen Token and Order Queue System is physically structured and how its components communicate with each other.

4.6.1 Main Components

- **Client Node:** The device used by customers, canteen staff, or admin to access the system through a web browser
- **Server Node:** Handles all system operations including order processing, token generation, and queue management
- **Database Node:** Stores and manages all data related to users, orders, menu items, and tokens

4.6.2 Deployment Structure

Client Node → Server Node → Database Node

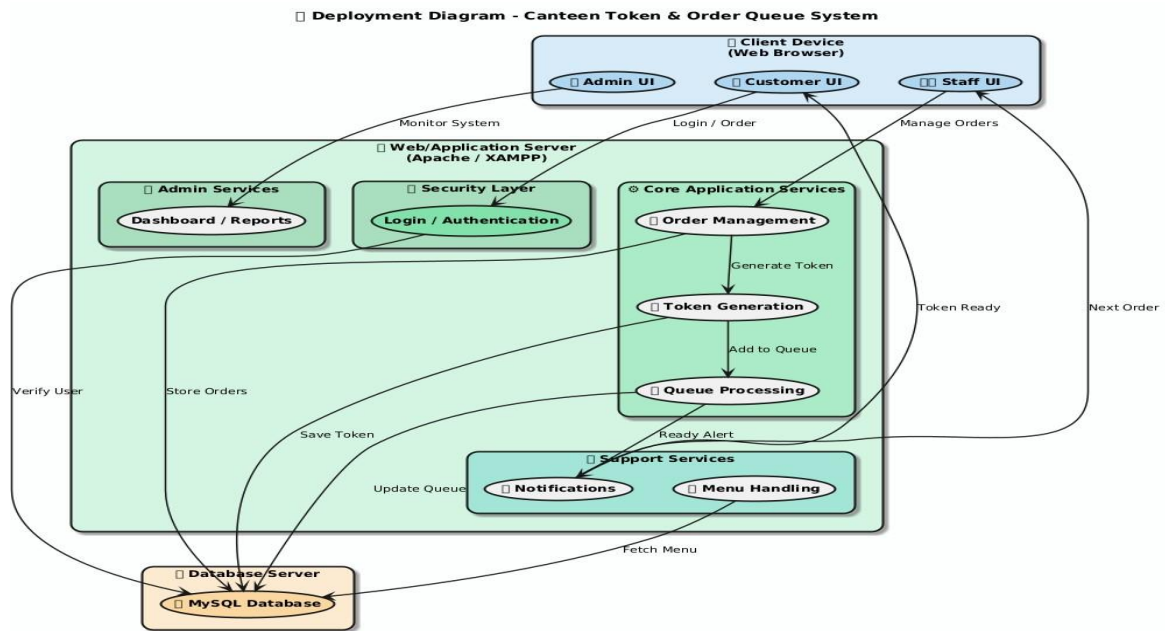
4.6.3 Communication

The client node sends user requests to the server node. The server processes the requests and communicates with the database node to store or retrieve the required data. The response is then returned to the client.

4.6.4 Description

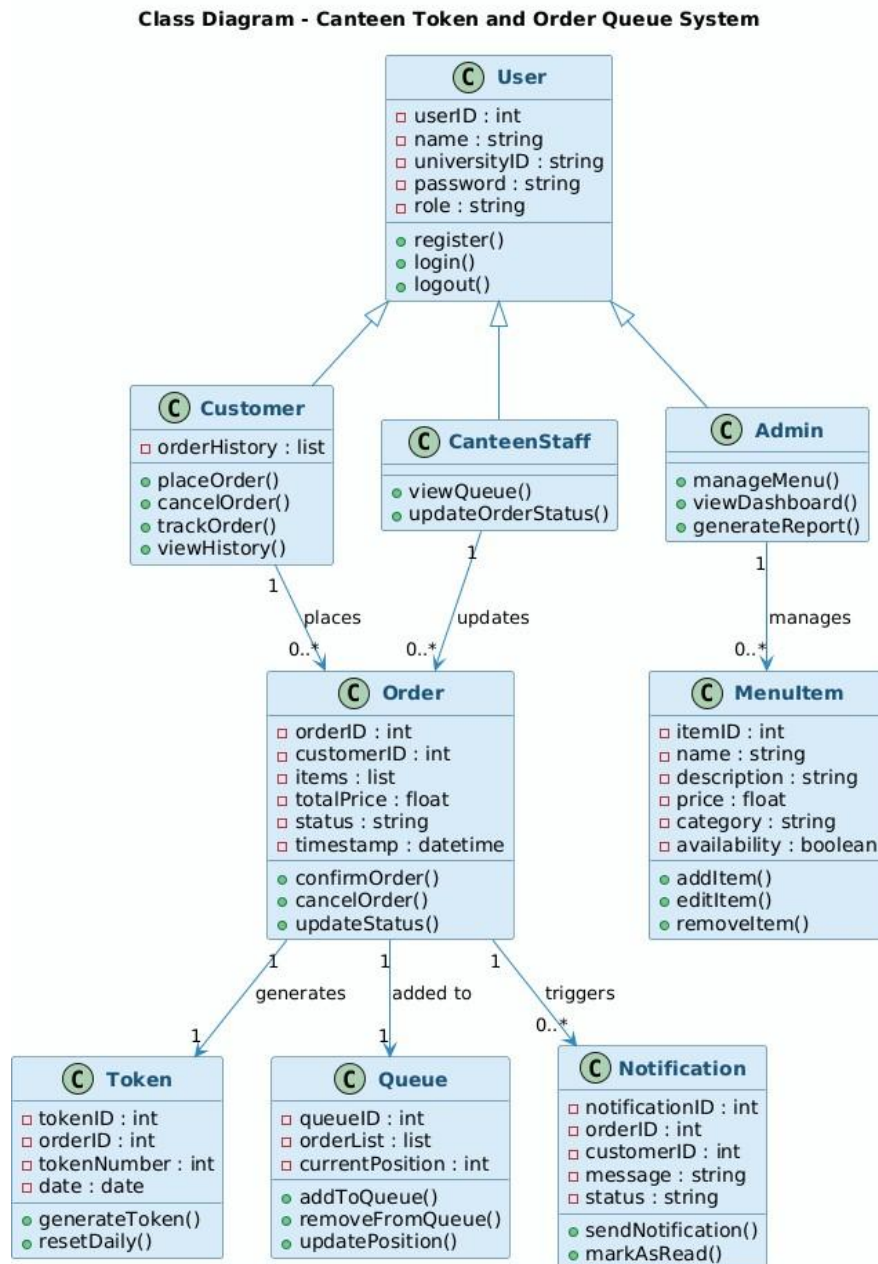
Every user action such as placing an order or updating an order status is initiated at the client node, processed at the server node, and recorded at the database node. The final result is displayed back to the user on their browser.

4.6.5 Diagram

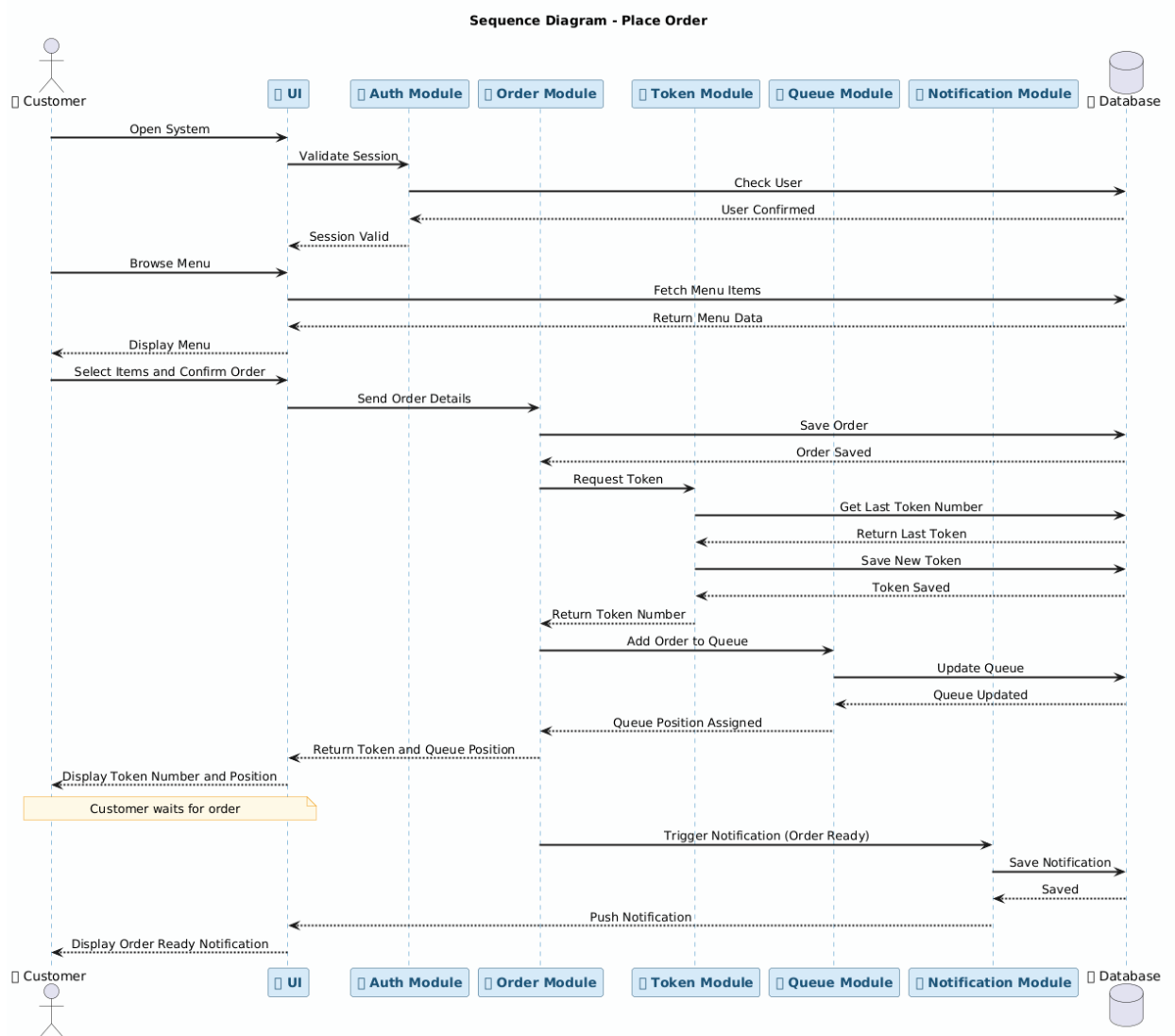


5. UML Diagrams

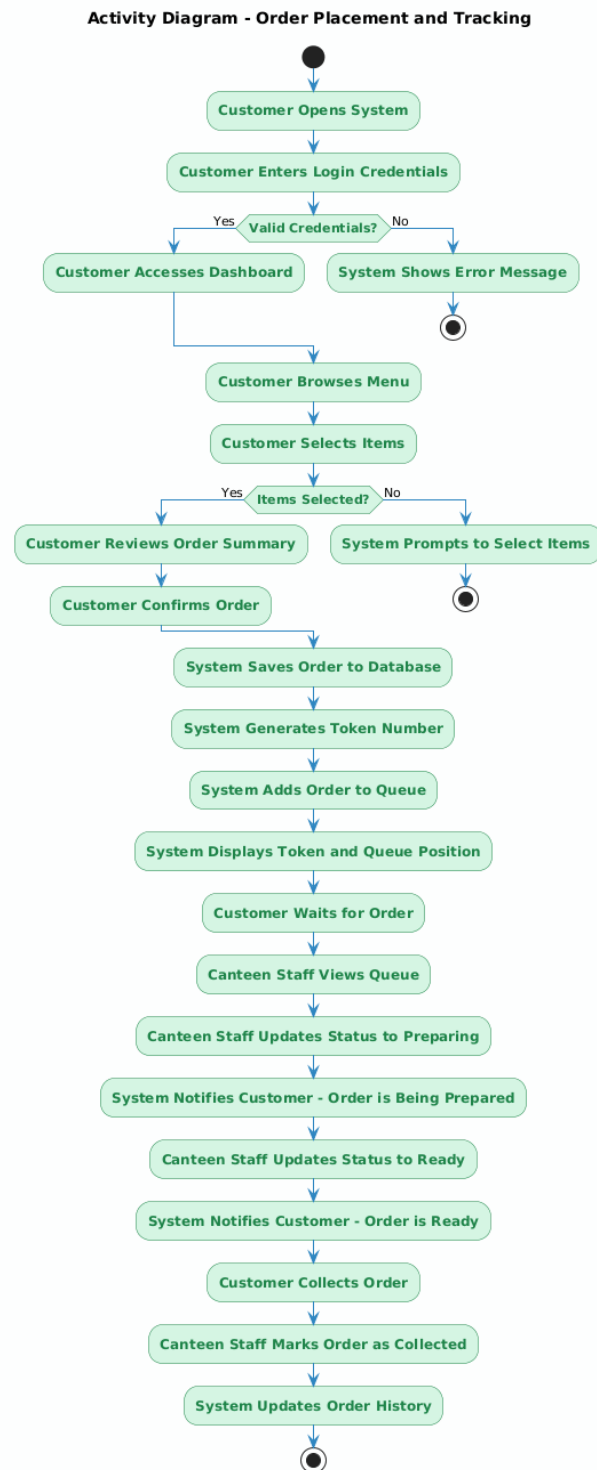
5.1 Class Diagram



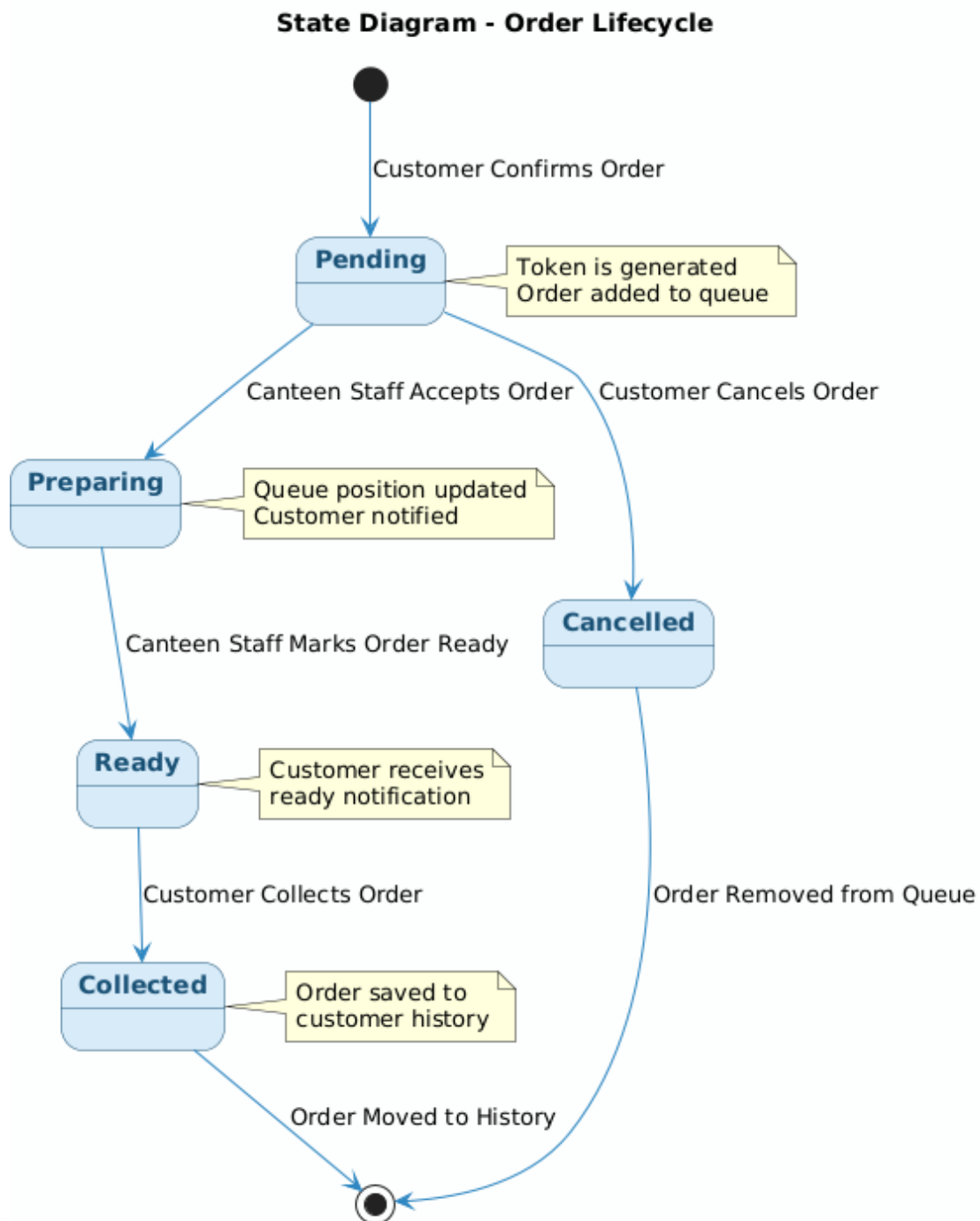
5.2 Sequence Diagram



5.3 Activity Diagram (Optional)



5.4 State Diagram (Optional)



6. Design Patterns

This section describes the three primary design patterns implemented in the Canteen Token and Order Queue System to ensure code maintainability, scalability, and adherence to architectural principles.

6.1 Singleton Pattern

- **Justification:** The system requires a centralized point for managing database connections to prevent memory leaks and redundant resource usage. Using the Singleton pattern for the DatabaseManager ensures that only one instance of the database connection is created and shared across the entire application.
 - **Implementation Approach:** The Database Manager class contains a private static instance and a private constructor. A public static method getInstance() is used by the Business Logic Layer to access the database operations safely without creating multiple connection objects.
-

6.2 Factory Pattern

- **Justification:** The system handles different types of users which are Customer, Canteen Staff, and Admin, each with distinct permissions and methods. The Factory pattern is used to centralize the creation logic for these user objects during the login process.
 - **Implementation Approach:** A UserFactory class is implemented with a createUser() method. Based on the role attribute retrieved from the database, the factory returns either a Customer, CanteenStaff, or Admin object. This hides the instantiation logic from the Presentation Layer and makes it easy to add future roles without modifying existing code.
-

6.3 Observer Pattern

- **Justification:** The system requires that customers are automatically notified whenever their order status changes. The Observer pattern is the natural fit here as it allows the order to notify all interested parties such as the customer interface and the notification module whenever a state change occurs.
- **Implementation Approach:** The Order class acts as the subject and maintains a list of observers. The Notification Module acts as the observer. Whenever the canteen staff updates the order status, the Order class calls the notify Observers() method which triggers the Notification Module to send an alert to the customer

7. Implementation Overview

This section outlines the technical foundation of the Canteen Token and Order Queue System, focusing on the technology stack, core features of the prototype, and the organizational structure of the source code.

7.1 Technology Stack

- **Frontend (Presentation Layer):** HTML, CSS, and JavaScript to build and style the web pages that users interact with. This provides a simple and accessible interface for customers, canteen staff, and admin through any web browser.
 - **Backend (Logic Layer):** PHP to handle form submissions, process orders, generate tokens, manage the queue, and interact with the database.
 - **Database (Data Layer):** MySQL to store and manage all system data including users, orders, menu items, tokens, and notifications.
 - **Development Tools:** Visual Studio Code for writing code, XAMPP for running the local Apache server and MySQL database, and phpMyAdmin for managing the database visually.
-

7.2 Prototype Features

The prototype focuses on the following core modules:

- **User Authentication:** A login page that identifies the user role and redirects them to their respective interface.
- **Menu Browsing and Order Placement:** A digital menu where customers select items and confirm their order through a simple web form.
- **Token Generation:** Automatic generation of a unique token number upon order confirmation, displayed immediately to the customer.
- **Order Status Management:** A canteen staff interface to view pending orders and update their status from Pending to Preparing to Ready to Collected.
- **Admin Dashboard:** A simple admin panel to add, edit, and remove menu items and view daily order counts.

7.3 System Structure / Code Organization

The code is organized into separate folders to reflect the Three-Tier Architecture and ensure a clean separation of concerns:

- **/views (Presentation Layer):** Contains all HTML and CSS files for the customer, canteen staff, and admin interfaces.
- **/controllers (Business Logic Layer):** Contains PHP files that handle user requests, process orders, generate tokens, and manage queue logic.
- **/database (Data Layer):** Contains the database connection file and SQL queries for all create, read, update, and delete operations.
- **/models (Entities):** Contains PHP files representing core data structures such as User, Order, MenuItem, Token, and Notification.

8. Testing Strategy

This section describes the testing approach adopted for the Canteen Token and Order Queue System. It covers the testing objectives, types of testing performed, sample test cases, and the tools used.

8.1 Testing Objectives

The main objectives of testing this system are:

- To verify that all functional requirements are correctly implemented and working as expected
- To ensure that the system handles invalid inputs and unexpected user actions gracefully
- To confirm that different user roles can only access features permitted to them
- To validate that the order queue follows the correct First In First Out sequence
- To ensure that token generation, order status updates, and notifications work correctly end to end

8.2 Types of Testing

- 1) **Unit Testing** Each individual module of the system is tested independently to verify that it performs its intended function correctly. For example the token generation module is tested to confirm it produces a unique sequential number for each order and resets at the start of each day.
- 2) **Integration Testing** The interaction between modules is tested to ensure they work correctly together. For example the integration between the Order Management Module and the Token Generation Module is tested to confirm that a token is always generated immediately after an order is confirmed.
- 3) **Functional Testing** The system is tested against each functional requirement to verify that all features work as specified. Each use case is executed and the output is compared against the expected result.
- 4) **User Acceptance Testing** The system is tested from the perspective of the end user to ensure it is easy to use and meets the needs of customers, canteen staff, and admin. This involves running through real world scenarios such as placing an order, tracking its status, and collecting it.
- 5) **Boundary Testing** The system is tested at the boundary conditions such as placing an order when the queue is empty, cancelling an order that is already in Preparing status, and logging in with incorrect credentials.

8.3 Test Cases

Test Case ID	Module	Test Case Description	Input	Expected Output	Status
TC-01	Authentication	Valid user login with correct credentials	Correct university ID and password	User redirected to their respective dashboard	Pass
TC-02	Authentication	Login attempt with incorrect password	Wrong password entered	System displays "Invalid ID or password"	Fail
TC-03	Order Placement	Place order with selected items	Customer selects items and confirms	Order added to queue and token number displayed	Pass
TC-04	Token Generation	Token generated after order confirmation	Confirmed order	Unique sequential token number generated and displayed	Pass
TC-05	Order Cancellation	Cancel an order in Pending status	Customer cancels Pending order	Order removed from queue and token invalidated	Pass
TC-06	Order Status Update	Canteen staff updates order status to Ready	Staff marks order as Ready	Status updated and customer receives notification	Pass
TC-07	Admin Menu Management	Admin adds new item with empty required fields	Admin saves without filling all fields	System displays validation error and does not save	Fail

9. Conclusion

9.1 Summary

The Canteen Token and Order Queue System was designed and documented as part of this Software Design and Architecture project. The system addresses the real world problem of disorganized and inefficient manual queuing in university canteens by providing a structured digital solution. Customers can place orders digitally, receive a unique token number, and track their order status in real time. Canteen staff manage incoming orders through a dedicated interface while the admin maintains full control over the menu and daily reports. The system was designed following the Three-Tier Architecture combined with the MVC pattern, and incorporates the Singleton, Factory, and Observer design patterns to ensure a maintainable, modular, and scalable solution.

9.2 Constraints and Limitations

The following limitations exist in the current version of the system:

- The system does not support online payment and relies on cash payment at the time of order collection
 - Notifications are limited to on-screen alerts only and do not support SMS or email
 - The system is designed for local network deployment and does not support cloud based access
 - Inventory and stock management for the canteen is outside the scope of this system
 - Reporting is limited to basic daily order counts without advanced analytics or trends
-

9.3 Recommendations for Future Work

The following improvements are suggested for future development:

- Integration of an online payment gateway to allow customers to pay at the time of order placement
- Addition of SMS and email notification support for customers not actively using the system
- Migration to a cloud based server to make the system accessible from anywhere on campus
- Development of a dedicated mobile application for faster and easier access
- Addition of an inventory management module to help canteen staff track stock levels
- Implementation of advanced analytics to identify peak hours, popular items, and daily trend

10. References

10.1 Books

1. Sommerville, I. (2022). *Engineering Software Products: An Introduction to Modern Software Engineering* (1st ed.). Pearson Education.
 2. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
-

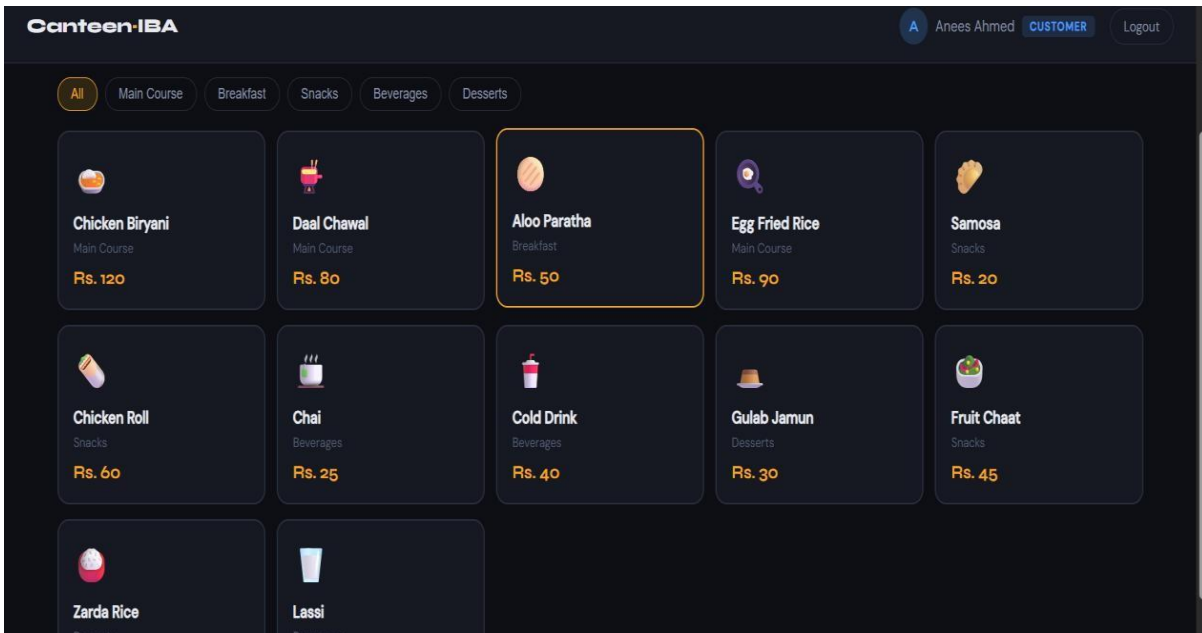
10.2 Tools

3. PlantUML Official Documentation. Retrieved from <https://plantuml.com>
4. MySQL Official Documentation. Retrieved from <https://dev.mysql.com/doc>
5. XAMPP Official Documentation. Retrieved from <https://www.apachefriends.org>
6. PHP Official Documentation. Retrieved from <https://www.php.net/docs.php>
7. Visual Studio Code. Retrieved from <https://code.visualstudio.com>

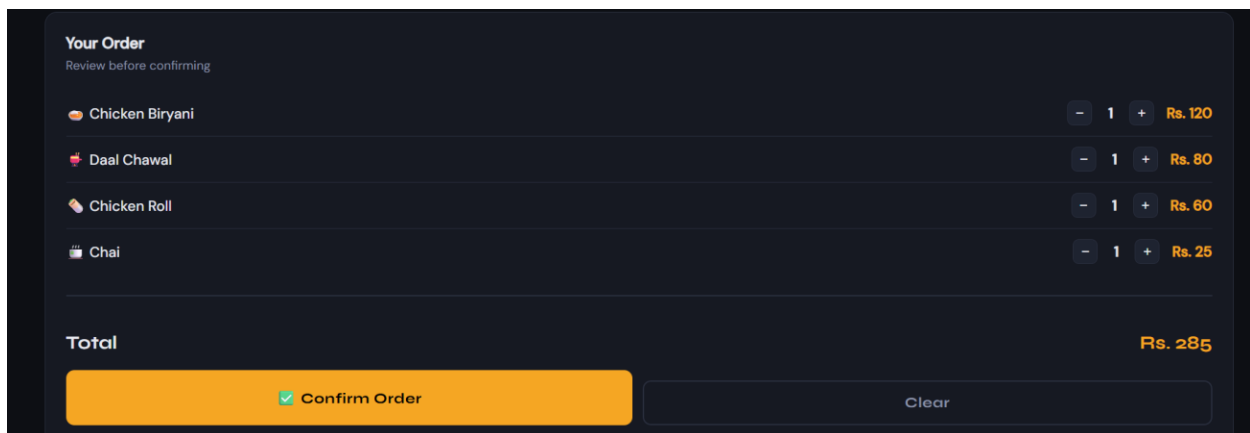
11. Appendices

11.1 User Interface

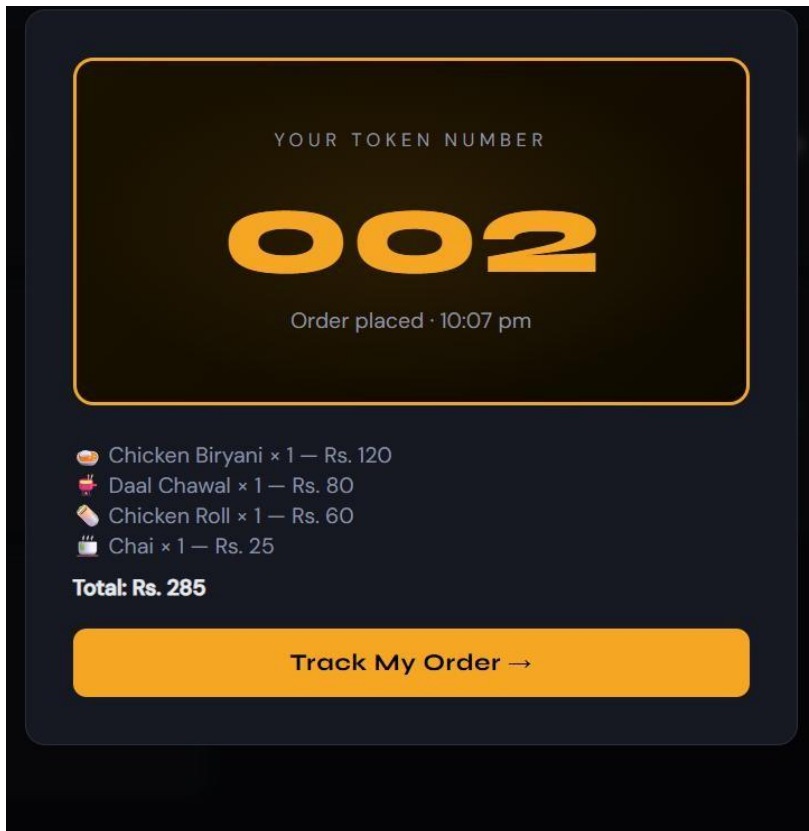
11.1.1 Menu



11.1.2 Order summary



11.1.3 Token generation



11.1.4 Admin panel

